

Java Inheritance Programs

Problem Statement, Description, Algorithm, UML Diagram, Code and Output

1. Single Inheritance

Problem Statement

Write a Java program to demonstrate single inheritance, where one subclass inherits properties and methods from one superclass. The program also demonstrates method overriding and method overloading.

Description

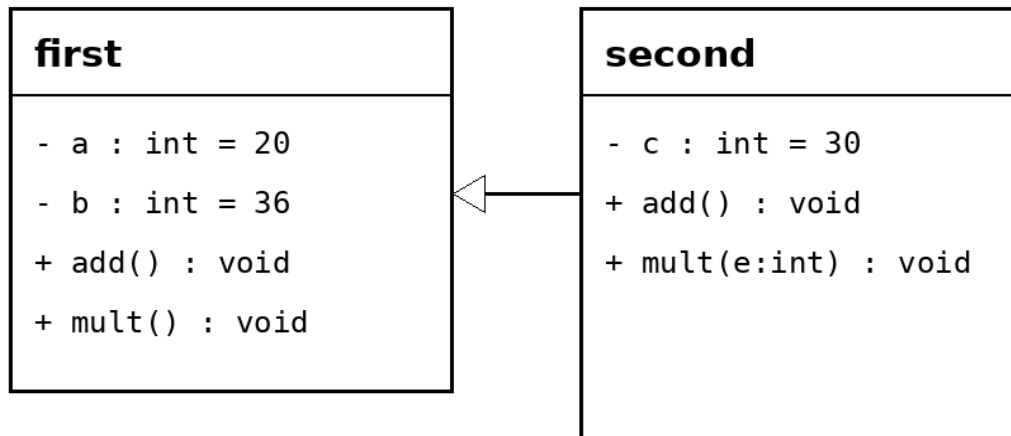
In this program, **first** is the parent class and **second** extends **first**. The parent class contains variables a and b and methods add() and mult(). The child class adds variable c, overrides add(), and overloads mult() using mult(int e).

Algorithm

1. Start the program.
2. Create parent class first.
3. Declare a = 20 and b = 36.
4. Define add() and mult() in first.
5. Create child class second using extends first.
6. Declare c = 30 and override add().
7. Define overloaded mult(int e).
8. Create an object of second.
9. Call add(), mult(35), and inherited mult().
10. Stop.

UML Diagram

Single Inheritance UML



Code

```
package inheritance;

class first {
    int a = 20;
    int b = 36;

    void add() {
        System.out.println("sum of the numbers:" + (a + b + 100));
    }

    void mult() {
        System.out.println("mult" + (a * 2));
    }
}

class second extends first {
    int c = 30;

    @Override
    void add() {
        System.out.println("sum of the numbers" + (a + b + c));
    }

    void mult(int e) {
        System.out.println("mult" + (e * 50));
    }
}

public class Single {
    public static void main(String[] args) {
        second sc = new second();
        sc.add();
        sc.mult(35);
        sc.mult();
    }
}
```

Output Screenshot

Single.java — Console Output

```
> java inheritance.Single  
sum of the numbers86  
mult1750  
mult40
```

2. Super Keyword

Problem Statement

Write a Java program to demonstrate the use of the super keyword to call methods of the parent class from the child class.

Description

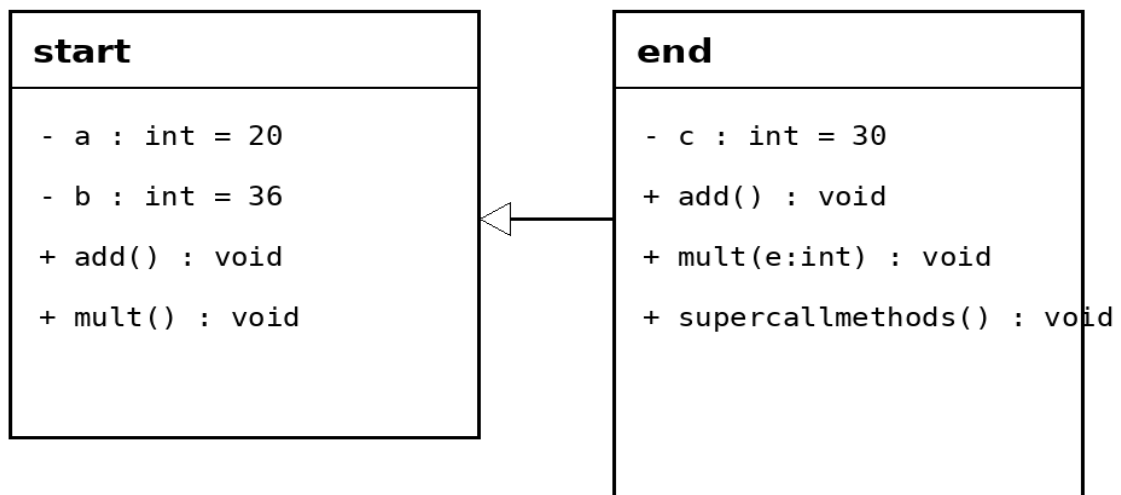
The program contains a parent class start and a child class end. The child class overrides add() and defines mult(int e). The supercallmethods() method uses super.add() and super.mult() to explicitly call the parent-class methods.

Algorithm

1. Start the program.
2. Create class start.
3. Declare a = 20 and b = 36.
4. Define add() and mult() in start.
5. Create class end extending start.
6. Declare c = 30.
7. Override add().
8. Define mult(int e).
9. Use super.add() and super.mult().
10. Create an object of end and call the methods.
11. Stop.

UML Diagram

super Keyword — UML



Code

```
package inheritance;

class start {
    int a = 20;
    int b = 36;

    void add() {
        System.out.println("sum of the numbers:" + (a + b + 100));
    }

    void mult() {
        System.out.println("mult" + (a * 2));
    }
}

class end extends start {
    int c = 30;

    void add() {
        System.out.println("sum of the numbers" + (a + b + c));
    }

    void mult(int e) {
        System.out.println("mult" + (e * 50));
    }

    void supercallmethods() {
        super.add();
        super.mult();
    }
}

public class Super {
    public static void main(String[] args) {
        end ex = new end();
        ex.supercallmethods();
        ex.add();
        ex.mult(35);
    }
}
```

Output Screenshot

Super.java — Console Output

```
> java inheritance.Super
sum of the numbers:156
mult40
sum of the numbers86
mult1750
```

3. Multilevel.java

Problem Statement

Write a Java program to demonstrate inheritance between classes one, two, and three and show method overriding/overloading as implemented in the supplied program.

Description

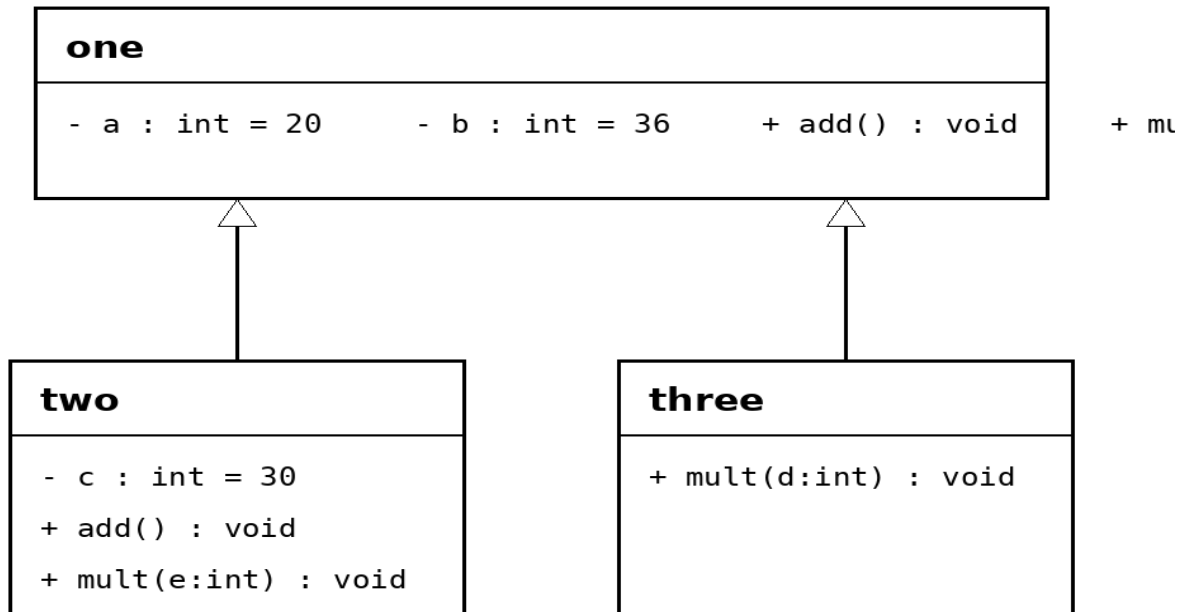
The supplied program defines class one with variables a and b and methods add() and mult(). Class two extends one and adds c, overrides add(), and defines mult(int e). Class three also extends one and defines mult(int d). Thus, as coded, two and three both directly extend one; this structure is hierarchical inheritance rather than multilevel inheritance.

Algorithm

1. Start the program.
2. Create class one.
3. Declare a = 20 and b = 36.
4. Define add() and mult().
5. Create class two extending one.
6. Declare c = 30.
7. Override add() and define mult(int e).
8. Create class three extending one.
9. Define mult(int d).
10. Create an object of two.
11. Call mult(5) and inherited mult().
12. Stop.

UML Diagram

Multilevel.java — UML (as coded)



Note: the supplied code has two and three both extending one

Code

```
package inheritance;

class one {
    int a = 20;
    int b = 36;

    void add() {
        System.out.println("sum of the numbers:" + (a + b + 100));
    }

    void mult() {
        System.out.println("mult" + (a * 2));
    }
}

class two extends one {
    int c = 30;

    void add() {
        System.out.println("sum of the numbers" + (a + b + c));
    }

    void mult(int e) {
        System.out.println("mult" + (e * 50));
    }
}

class three extends one {
    void mult(int d) {
        System.out.println("mult" + (a * b * 2));
    }
}

public class Multilevel {
    public static void main(String[] args) {
        two kc = new two();
        kc.mult(5);
        kc.mult();
    }
}
```

Output Screenshot

Multilevel.java — Console Output

```
> java inheritance.Multilevel  
mult250  
mult40
```